The Ranke Graph

Attributed Claims as a Data Structure

An abstract data type for provenance-first knowledge. Nodes, edges, and the invariants that make the DAG a Merkle-integrity witness. No implementation details, no stack choices, no code.

Abstract

We define the Ranke graph: an abstract data type for append-only, content-addressed knowledge in which provenance is the foundational data structure rather than metadata attached to it. Nodes carry opaque content; edges are partitioned by `class` into a provenance subgraph (acyclic, Merkle-secured) and a semantic subgraph (potentially cyclic). A node's identity is a cryptographic hash of its content, its atomically-created edges, and its contributor attribution, so each node witnesses the integrity of its entire provenance subgraph and writes are idempotent by construction. A snapshot is a special node whose inputs are the current heads and the previous snapshot; snapshot IDs form a hashchain publishable to any external timestamping service. This paper specifies the ADT and proves its invariants.

1. Introduction

Consider three statements:

> *Alice likes apples.*
>
> *Alice wrote Bob an email saying she likes apples.*
>
> *A file exists, attributed to Alice by its headers, that appears to be a copy of an email to Bob in which Alice claims to like apples.*

The first is a claim about the world. The second adds an attribution. The third is an observation of existence — a file is present, with the stated bytes, metadata and content.

Storing at the first layer is the classic goal of database design. Schemas, integrity constraints, and transactions are built to maintain a consistent model of the world; the caller is expected to have applied sound epistemology before writing data. When facts change, or sources disagree, the database is edited to align; its earlier states, and the disagreement itself, are discarded. In database discipline this is called *destructive consolidation* or *last-write-wins*; it is usually filed under *data cleaning*, and treated as consistency rather than loss.

This is the ordinary condition of the enterprise data store, and works so long as the caller supplies correct facts.

The Ranke graph is designed for the opposite stance. It stores only at the third layer. Every node is an observation-of-existence: this artifact, with these bytes, this attribution, added to the graph at this moment. The graph does not record whether Alice likes apples, or whether she wrote the email. It records that a file is present, its metadata is as given, and the record has not been altered since it was written. The guarantee is narrower than a conventional database's, and therefore keepable.

We call what the graph stores an *attributed claim*. A claim, in this paper, is not a proposition asserted as true; it is a preserved, attributed record of the form *"actor X produced this content at time T, derived from these inputs."* The claim and its attribution are stored as a single unit. Neither is verified nor refuted by the data type itself; both are preserved as received, immutable once written, traceable to every input

that contributed to them. Derivations — extracts, summaries, conclusions — are constructed on top of the graph by its contributors, with their own stake and context.

The Ranke graph is not a pure graph-theoretic construction; it is a structure shaped by a purpose. Without that purpose — preserving attributed claims without climbing the epistemic ladder — the features that follow would be arbitrary.

This paper defines the Ranke graph as an abstract data type — the minimum contract an implementation must satisfy. A reference implementation will be the subject of a follow up paper.

2. The Problem: Knowledge Without Provenance

2.1. The Archival Tradition

The Ranke graph takes its name from Leopold von Ranke (1795–1886), the historian who transformed his discipline by insisting that every historical claim must trace back to a critically examined primary source. Ranke's famous phrase — history *"wie es eigentlich gewesen"*, "as it actually was" — has since been rightly criticized for assuming unmediated access to past reality. The data type is built on that criticism: the primary data point is never *"how it was"* but the artifact that reports, claims, or interprets it — an email, a photo, an audio recoding, a document. What a Ranke graph stores is always someone's utterance about the world, never the world itself. What survives from Ranke's method, intact, is the discipline of attribution: nothing is asserted without its derivation, and nothing is derived without linking back to its sources.

2.2. The CS Priority That Was Never Operationalized

Existing systems that address this tension do so partially. Temporal knowledge graphs (Graphiti/Zep, Rasmussen 2025) preserve *when* facts were valid but perform destructive entity summary updates, losing derivation history. Versioned knowledge bases (TerminusDB, Mendel-Gleason et al.) track *what* changed across snapshots but not *why* or *how* knowledge was derived. Immutable databases (Datomic, Hickey 2012 Fluree) preserve all historical states but lack a semantic knowledge layer and do not model derivation chains. No existing system treats the full chain of provenance — from raw source artifact through extraction, normalization, and synthesis — as first-class, queryable knowledge.

3. Philosophical Grounding

(Quotes and design notes collected in `papers/01-rankedb/quotes.md`. Key points for P0:)

3.1. Provenance and consensus are orthogonal problems

- **Provenance** = attribution (who said what, when, on what basis). Solvable by construction.
- **Consensus** = social agreement on what to trust. Human process, not database architecture.
- The Ranke graph handles provenance. Consensus is downstream, built by contributors on top.

3.2. Bounded scope: personal to small-enterprise

- At bounded scale, trust is pre-established, ontology is finite, adversarial resistance is simple.
- The Ranke graph does not aim for Wikipedia-scale global consensus.

3.3. Thesis

The Ranke graph stores attributed claims; common truth is what contributors build on top when they want it.

4. Core Properties

The properties below are what a Ranke graph guarantees by construction. Vocabulary used without further comment: a single graph of nodes connected by typed edges; every node carries provenance edges back through its inputs to the sources it was derived from.

4.1. Everything Is Knowledge

The Ranke graph makes no distinction between data, metadata, and provenance. Every claim made *about* the graph is itself a node in the graph, with its own provenance:

- a classification ("this node belongs to the finance domain"),
- a summary ("this is a condensed version of the conversation at node X"),
- an alias ("this node refers to the same person as node Y"),
- a creation record ("this node was added by contributor X with configuration Y").

This principle — *everything is knowledge* — eliminates the need for separate metadata systems, tagging taxonomies, or logs of contributor activity as additional infrastructure: all of these are expressible as nodes in the graph, derived from the same sources, subject to the same provenance and immutability guarantees, and queryable through a single API.

From this ontological flatness follows a structural claim. If every claim is a node with provenance, then what is the *primary* content of the system? In conventional knowledge graphs, claims are primary and provenance is an annotation layer bolted on top — an afterthought that explains where things came from. No such split exists in the Ranke graph.

Provenance is not an annotation on the knowledge — it *is* the knowledge. Every derivation, every thought, every projected fact is itself a node in the graph, linked to the inputs it was derived from. There is no "real" layer above and a "provenance" layer below; it is one graph, and the knowledge and its derivation are stored together.

This inversion has concrete operational consequences: operations that would require complex graph surgery in a conventional system become simple view operations in the Ranke graph. Reprocessing sources with better tools produces new nodes alongside old ones — no migration required. Filtering out results from an obsolete contributor is a query parameter, not a data operation. Evaluating competing interpretations of the same source is a traversal, not a diff between snapshots.

4.2. Immutability and Accumulation

The Ranke graph is strictly append-only. No node or edge is ever modified or deleted through runtime operations. When new information contradicts existing knowledge, the contradiction is represented as a new node — not as an update to the old one. Both coexist in the graph, each with full provenance.

Contradiction is not a bug to resolve, it is a fact about the evidence base. Resolving it destroys information: the consolidated graph holds strictly less knowledge than the contradictory one it replaced.

This design is a deliberate bet on the trajectory of language model context windows. Systems that destructively consolidate today — merging entity summaries, deduplicating facts, compacting histories — optimize for current retrieval efficiency at the cost of inferential depth. The Ranke graph is built for a future in which a model able to traverse the full derivation history of a belief as needed — contradictions, revisions, and competing interpretations — may produce better reasoning than one given only a consolidated summary.

5. Nodes

A node is the unit of knowledge in the graph. The minimal definition:

```
node = {
  type:           string,
  content:        bytes,
  created_at:     timestamp,
  contributor_id: identity of the contributor,
  edges:          set of edge ids created with the node,
  fields_0..n:    extension fields (implementation-defined; may be empty)
}
```

TODO: contributor_id has a chicken-egg-problem… we need a root contributor that seeds the DB

A node's identity is the cryptographic hash of its record:

$$\mathrm{id}(v) \coloneqq H(\mathrm{type}(v) \parallel \mathrm{content}(v) \parallel \mathrm{id}(e_1) \parallel \ldots \parallel \mathrm{id}(e_n) \parallel \mathrm{created\_at}(v) \parallel \mathrm{contributor\_id}(v) \parallel \mathrm{fields}_0(v) \parallel \ldots \parallel \mathrm{fields}_n(v))$$

where $e_1, \ldots, e_n$ are the edges created with v (§ Edges). Two nodes with identical content but different provenance — different edges, different contributor, or different extension fields — produce different ids and are therefore distinct nodes.

- `type` is a plain string; the ADT prescribes no vocabulary.
- `content` is opaque bytes. Interpretation is the concern of contributors, not the ADT.
- `fields_0..n` is a placeholder for any number of additional fields. Every field participates in $\mathrm{id}(v)$; the Merkle property (§ Merkle integrity) applies to all fields, not just the ones named here.

6. Edges

An edge belongs to exactly one node: the node that contains its id in its `edges` set. We call that node the edge's *parent*; the parent is recoverable by traversal, not stored in the edge itself.

```
edge = {
  target:      hash_of_target_node,
  class:       provenance | semantic,
  type:        kind of relation (e.g. "family", "derivation", "ownership"),
  content:     the relation itself (e.g. "might be the lost brother of",
"summarized_to"),
  direction:   parent_to_target | target_to_parent,
  fields_0..n: extension fields (implementation-defined; may be empty)
}
```

An edge's identity is the cryptographic hash of its record:

$$\mathrm{id}(e) \coloneqq H(\mathrm{target}(e) \parallel \mathrm{class}(e) \parallel \mathrm{type}(e) \parallel \mathrm{content}(e) \parallel \mathrm{direction}(e) \parallel \mathrm{fields}_0(e) \parallel \ldots \parallel \mathrm{fields}_n(e))$$

Omitting the parent from the record is not cosmetic: if parent were a field, $\mathrm{id}(e)$ would depend on $\mathrm{id}(v)$, which in turn depends on $\mathrm{id}(e)$ through the node's `edges` set, and no consistent identity assignment would exist. Keeping the parent implicit resolves the recursion.

The `direction` field records which way the *semantic* arrow of the edge points between its (implicit) parent and its `target`. Without it, the ADT could not express, for example, that an older node stands in a relation to a newer one: the older node cannot create the edge (it already existed), so the newer node must — but the semantic relation may still read as "older → newer."

The split between `type` and `content` is the same as for nodes: `type` classifies (what *kind* of relation this is), `content` carries the specific assertion.

- **Provenance edge (class = provenance):** the parent was derived from target. `direction` is always `target_to_parent` — derivation flows from the earlier node into the one that cites it. Acyclic by construction (§ DAG property under circular semantics).
- **Semantic edge (class = semantic):** the parent asserts something about target. `direction` may be either way. `type` classifies the relation, `content` carries the specific assertion, `direction` tells the reader how to read it.
- `fields_0..n` is a placeholder for any number of additional fields the edge can carry. Every field participates in $\mathrm{id}(e)$.

A node carries the ids of its edges in its own record, which makes those ids part of the node's id (§ Merkle integrity); edges are therefore Merkle-secured through the parent node that created them.

The `class` field separates the provenance subgraph (acyclic, Merkle-secured) from the semantic subgraph (potentially cyclic, expressive). Both coexist in the same graph, both are immutable once created, and both contribute to the parent node's identity through their ids.

7. Node Creation is Atomic

A node and all its edges are created in a single atomic transaction:

- n provenance-class edges (derivation from sources and prior derivations),
- m semantic-class edges (relations asserted),
- 1 content payload,
- 1 contributor attribution.

Nothing can be added to a node after creation. No edge can be added later. The node's hash covers everything it will ever have. This is what makes the Merkle property hold: $h(v)$ is final at creation time.

8. Hash Function Agnosticism

P0 uses $H(x)$ to denote a cryptographic hash function, without committing to a specific algorithm. The ADT allows:

- A hash-type prefix on ids: `sha256:a3f2b7c...`
- Coexistence of different hash functions during migration
- Future migration to post-quantum hash functions

Reference: Haber & Stornetta (1991), "How to Time-Stamp a Digital Document" — the foundational paper on cryptographic timestamping. They demonstrated the concept by publishing hash digests in the New York Times.

9. Formal Results

9.1. DAG property under circular semantics

Let $G = \big(V, E_{\text{prov}} \cup E_{\text{sem}}\big)$ be the graph, where edges are partitioned by the `class` field into provenance-class and semantic-class edges. Every edge e has a target (a node it points at) and an implicit parent (the node whose `edges` set contains $\mathrm{id}(e)$). Edges are created atomically with their parent node.

- **Provenance edge (class = provenance):** "my parent was derived from target."
- **Semantic edge (class = semantic):** "my parent asserts something about target."

Define the provenance subgraph $G_p = \big(V, E_{\text{prov}}\big)$.

Theorem. G_p is acyclic.

Proof. Every node v has a creation time $t(v)$. Provenance edges can only target nodes that existed before v was created: for every edge $e \in E_{\text{prov}}$ with parent v, $t(\text{target}(e)) < t(v)$. This establishes a strict partial order on V by creation time. A strict partial order admits no cycles. ■

Semantic edges are not subject to this constraint — they can target any existing node, including "older" neighbors. Therefore G may contain cycles (through semantic edges), but G_p cannot.

Corollary. The provenance subgraph G_p is always a DAG, regardless of the semantic richness of the other edges. Circular semantics (A knows B, B knows A) are modeled by two separate relation nodes, each with semantic edges, but the provenance subgraph remains acyclic.

9.2. Merkle integrity

Every ID in the system is a cryptographic hash H.

Edge hash:

$$h(e) = H(\text{target}(e) \parallel \text{class}(e) \\ \parallel \text{type}(e) \parallel \text{content}(e) \parallel \text{direction}(e) \\ \parallel \text{fields}_0(e) \parallel \ldots \parallel \text{fields}_n(e))$$

Node hash:

$$h(v) = H(\text{type}(v) \parallel \text{content}(v) \\ \parallel h(e_1) \parallel \ldots \parallel h(e_n) \\ \parallel \text{created\_at}(v) \parallel \text{contributor\_id}(v) \\ \parallel \text{fields}_0(v) \parallel \ldots \parallel \text{fields}_n(v))$$

where $e_1, \ldots, e_n$ are all edges (provenance- and semantic-class) created with v, and $\text{fields}_0, \ldots, \text{fields}_n$ are the extension fields added by any implementation that refines P0.

Theorem. Manipulation of any node v' in the provenance subgraph of v changes $h(v)$.

Proof. By induction on the depth of the DAG.

Base case: $v' = v$. Changing any field of v changes $h(v)$ directly (H is collision-resistant). ✓

Inductive step: v' is an ancestor of v in G_p. There exists a path $v' \to \ldots \to u \to v$ in G_p (following provenance edges). By inductive hypothesis, manipulation of v' changes $h(u)$. $h(u)$ is the target hash used in the computation of some provenance edge e of v. Changing $h(u)$ changes $h(e)$. Changing $h(e)$ changes $h(v)$ (since $h(e)$ is part of v's hash computation and H is collision-resistant). ■

Corollary. Each node hash witnesses the integrity of its entire provenance subgraph. Tampering anywhere below is detectable at the root.

9.3. Content-addressing and idempotency

Theorem. Identical content with identical provenance produces identical node hashes.

$$\forall v_1, v_2 : \text{fields}(v_1) = \text{fields}(v_2) \to h(v_1) = h(v_2)$$

Since node ID = node hash, identical nodes are the same node. Writes are idempotent by construction. Deduplication is free.

9.4. Snapshot hashchain

Snapshots are special nodes whose inputs are all current heads (nodes with no children in G_p) plus the previous snapshot.

$$s_0 = H\big(\text{heads}\big(G_p, t_0\big)\big)$$
$$s_n = H\big(\text{heads}\big(G_p, t_n\big) \parallel s_{n-1}\big)$$

The snapshot sequence $(s_0, s_1, ..., s_n)$ is a hashchain. Each snapshot witnesses the graph state AND all previous snapshots. Manipulation of any s_i invalidates all s_j for $j > i$.

Snapshot hashes can be published to any external timestamping service — e.g. in the New York Times or a public ledger (Haber & Stornetta, 1991) — to provide third-party proof of graph state at a given point in time.

10. Auth-Scoped Visibility and Merkle Compatibility

Auth-scoped visibility (a node derived from a confidential source is automatically confidential) is compatible with the Merkle-DAG.

A user receives a verifiable subgraph: full nodes with content for everything in scope. For branches outside their scope, they see only the hash — enough to verify the integrity of their own subgraph, but no content access.

```
[hash_only] ← confidential node, user sees only hash
     ↓
[full node] ← derived, user has access
     ↓
[full node] ← user has access
```

The user can verify: "my subgraph is intact, it builds on a node with hash X whose content I don't know." Integrity is provable without transparency. Only the server sees everything.

Merkle structure is what *enables* verifiable partial views. Auth scoping and Merkle integrity are complementary.

11. Paper Structure (planned)

Paper	Title	Scope
P0	The Ranke Graph	Abstract data type, invariants, formal proofs, philosophy
P1	RankeDB	Reference stack, dark S3, caching, branching, snapshots, compliance
P2	Workers	Pipeline, dispatch, reactive/analytical, claim decomposition
P3	Retrieval	Memory agents, verification gate, conviction, user confirmation
P4	Chat Frontend	Stacker, multi-agent coordination, user interface